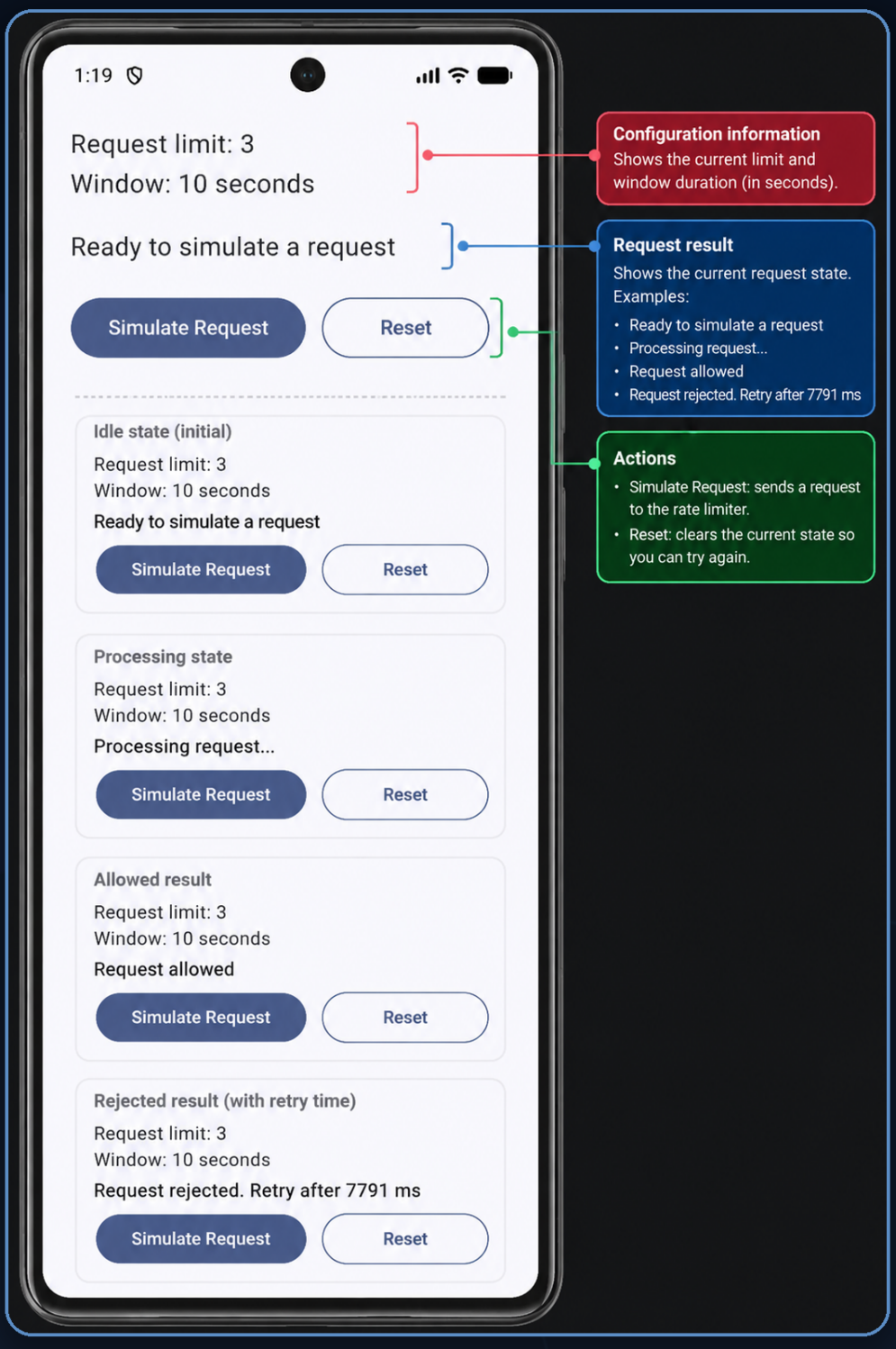
ANDROID ENGINEERING • PROJECT NOTES

RATE LIMITER PLAYGROUND

Senior Android Engineer

Interview guide • Design, trade-offs and testing



Kotlin · Coroutines · StateFlow · Hilt · Unit Testing

Lessons from building a working Android prototype

Contents

1. Explain the project in 30–60 seconds 3
Visual record: design and actual app outcomes 3
2. Architecture and responsibilities 5
3. Rate-limiting algorithm 6
4. State safety and coroutine concurrency 6
5. Time abstraction and monotonic clocks 7
6. Generic algorithm versus runtime owner 8
7. Coordinator, configuration replacement, and reset semantics 8
8. ViewModel state, MVI-style intents, and StateFlow 9
9. Version counters and stale results 9
10. Dependency Injection and Hilt 10
11. Unit testing strategy 11
12. Common interview questions and strong answer outlines 12
13. Things to avoid saying in an interview 13
14. Final project review checklist 13
15. Suggested README section 14
Main takeaway 14

This document summarizes the important engineering concepts practiced while building the Rate Limiter Playground. Use it to prepare for system-design and Android technical interviews, and keep it in the project as a record of the decisions and trade-offs.

> **Project summary:** A small Android app that lets a user configure a fixed-window rate limiter, simulate requests, inspect allow/reject decisions, and reset runtime state. The project uses Kotlin, coroutines, `Mutex`, `StateFlow`, Jetpack Compose, a ViewModel, dependency injection with Hilt, and unit tests.

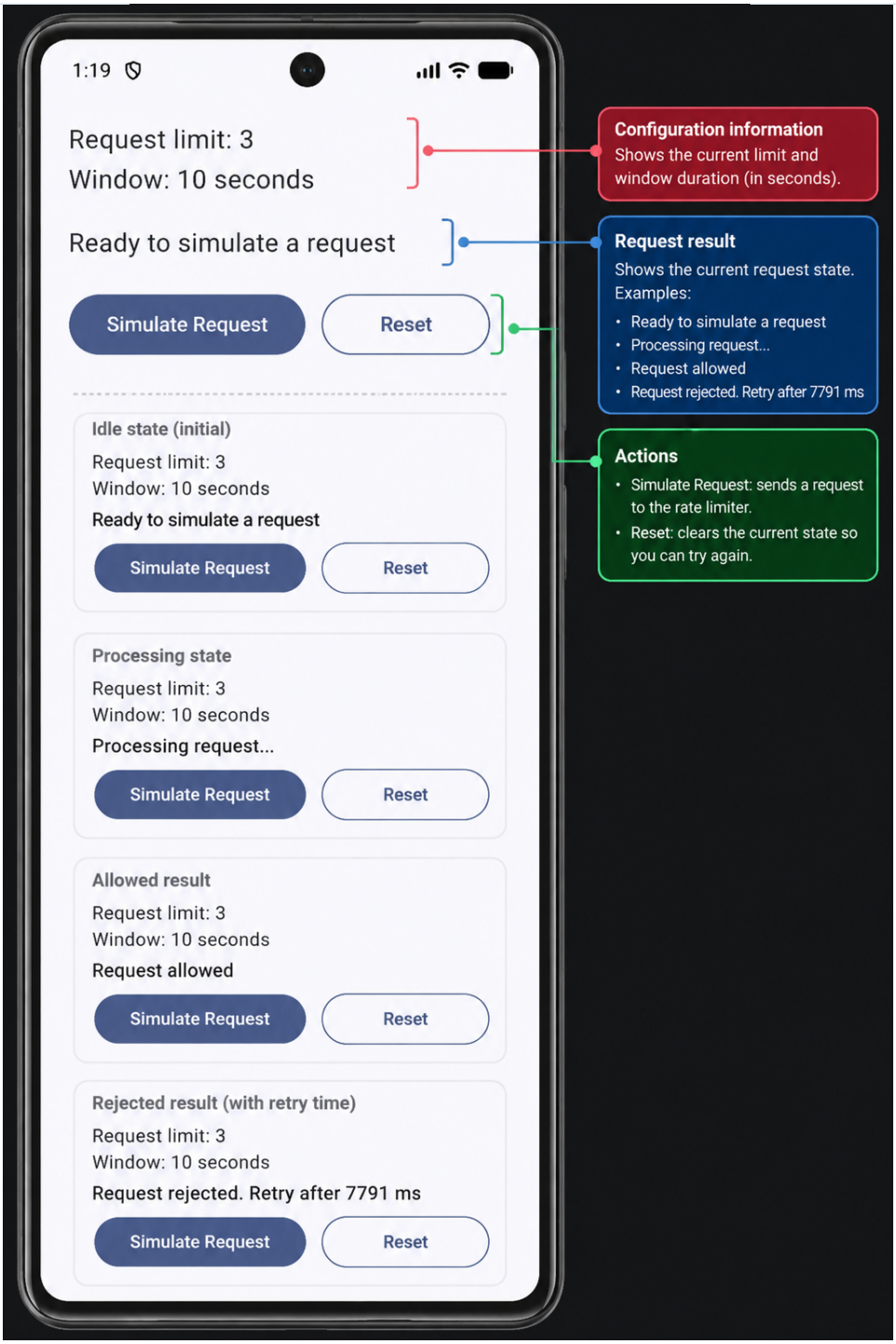
1. Explain the project in 30–60 seconds

“I built a Rate Limiter Playground to explore how to implement a fixed-window rate limiter and connect it to an Android UI safely. The domain layer contains the rate-limiting algorithm and decision models. A `RateLimiter` serializes state evaluation and updates so concurrent requests cannot exceed the configured limit. A coordinator owns the active limiter and handles configuration replacement and reset. The UI uses Compose, a ViewModel, intents, and `StateFlow`. Time is abstracted behind a `TimeProvider` so the algorithm can be tested deterministically. Hilt assembles the application dependencies, while unit tests cover the algorithm, concurrency, UI state transitions, configuration validation, and reset behavior.”

Be ready to explain the choices below and their limitations rather than simply listing technologies.

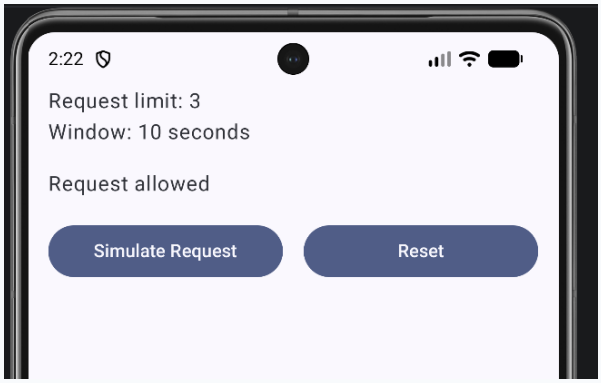
Visual record: design and actual app outcomes

The images in this section preserve both the original UI concept and examples of the running prototype. The design mockup communicates the intended controls and result states; it is a design reference, not a claim that every detail is identical to the implemented screen. The two app screenshots are real development captures illustrating an allowed request and a rejected request with a retry estimate. They show the prototype at the time of capture, including the emulator framing and crop.



Design mockup showing the configuration, request result, Simulate Request, and Reset controls

Design reference — intended configuration and request-result states.



Allowed request: the app displays “Request allowed” for the current configuration.



Rejected request: the app displays a retry-after estimate when the limit is exhausted.

These visuals are useful interview evidence because they connect the architecture to observable behavior: configuration is shown to the user, requests produce explicit decisions, and rejection communicates when another attempt may be appropriate. Avoid implying that screenshots alone prove correctness; unit tests and a clearly defined algorithm establish that.

---

2. Architecture and responsibilities

The project separates code by responsibility:

- **`domain/`**: rate-limit configuration, decisions, algorithm/state, `RateLimiter`, and the `TimeProvider` abstraction. This layer should contain the core rules and avoid Android UI dependencies.
- **`application/`**: `RateLimiterCoordinator`, which manages which limiter is active and coordinates configuration changes/reset.
- **`platform/`**: Android-specific implementation of abstractions, such as `AndroidTimeProvider` using `SystemClock.elapsedRealtime()`.
- **`ui/ratelimiter/screen/`**: Compose rendering and UI route/lifecycle collection.
- **`ui/ratelimiter/state/`**: UI state and intents that describe user actions.
- **`ui/ratelimiter/viewmodel/`**: intent handling, validation, coroutine orchestration, and publishing UI state.
- **`di/`**: Hilt modules that provide interface bindings and dependencies requiring custom construction.

Why separate these layers?

Each component has a narrower reason to change. The algorithm can be tested without Compose. Android clock access can be replaced with a fake. The ViewModel can be tested without launching Hilt. The UI describes state and forwards actions rather than implementing the rate-limiting algorithm.

Interview question: Is this Clean Architecture?

A good answer: “It follows some Clean Architecture ideas—domain rules are separated from Android UI/platform concerns, and dependencies are supplied from outside. I would call it a lightweight layered architecture rather than claim a full formal Clean Architecture implementation. For this small app, extra use-case classes or Gradle modules would add little value unless complexity justified them.”

3. Rate-limiting algorithm

Fixed-window behavior

A fixed-window limiter groups requests into time windows. In a window of duration `windowMillis`, at most `limit` requests are allowed. When a new window begins, the request count is reset for that window.

Example with `limit = 3` and a 10-second window:

- Requests 1, 2, and 3 in the same window: allowed.
- Request 4 in that window: rejected.
- Once a new window begins: the allowance resets.

Advantages

- Simple to explain and implement.
- Low memory usage: only a small amount of window/count state is needed.
- Fast request evaluation.

Limitations and trade-offs

- **Boundary burst:** fixed windows can allow a burst around a window boundary. For example, a client could use the full allowance just before a boundary and another full allowance just after it.
- **Not a rolling limit:** it does not guarantee that no more than `limit` requests occur in every arbitrary interval of length `windowMillis`.
- **Not distributed:** this in-memory implementation enforces a limit within one active limiter/process. Multiple app processes or servers would need shared/distributed coordination if a global limit were required.

Be able to compare this with a sliding-window log, sliding-window counter, and token bucket. Don’t claim fixed window is universally best; it is appropriate when simplicity and low overhead matter more than smoothing boundary bursts.

4. State safety and coroutine concurrency

Why protect the critical section?

The important operation is not just incrementing a counter. It is the combined sequence:

1. Read the current limiter state.
2. Evaluate the request against the current time and configuration.
3. Commit the returned new state.
4. Return the corresponding decision.

These steps must be atomic with respect to concurrent calls to the same limiter. Otherwise, two coroutines could both read the same old count and both decide that a request is allowed, exceeding the configured limit.

The `RateLimiter` uses a coroutine `Mutex` to serialize that state transition. A mutex protects mutable state; it does not itself define the rate-limiting algorithm.

What `Mutex` does and does not do

- It ensures only one coroutine at a time enters the protected critical section for that mutex.
- `withLock` releases the lock when the block finishes or throws.
- It does not automatically make unrelated state thread-safe.
- It does not replace correct state ownership or a clear definition of when a configuration change takes effect.

Why not just use `synchronized`?

A coroutine `Mutex` is designed for coroutine coordination and suspends a waiting coroutine instead of blocking its thread while waiting for the lock. `synchronized` is a JVM monitor that blocks the calling thread. Use the right mechanism for the concurrency model; do not hold a blocking lock around suspending work.

Interview question: Why is a concurrent test important?

A sequential test can pass even if the limiter has a race. A concurrency test launches multiple requests concurrently and verifies the invariant—for example, 100 simultaneous requests with a limit of 1 should result in exactly 1 allowed decision and 99 rejected decisions for the same window. The test must genuinely create overlap, not merely loop sequentially.

---

5. Time abstraction and monotonic clocks

The domain depends on a `TimeProvider` rather than directly reading Android's clock. The Android implementation uses `SystemClock.elapsedRealtime()`.

Why inject time?

- Tests can use a fake clock and advance time deterministically.
- Tests do not need to sleep or depend on wall-clock timing.
- Domain code is not coupled directly to Android APIs.

Why `elapsedRealtime()` instead of wall-clock time?

For elapsed durations, a monotonic clock is the right model. Wall-clock time can change due to user adjustments or clock synchronization. A monotonic elapsed-time source is intended for measuring time intervals and does not jump with civil-time changes.

Interview-ready answer

"I abstracted time because the algorithm depends on elapsed time. That gives tests a deterministic clock and isolates Android-specific time access behind an interface. I use monotonic elapsed time instead of calendar time because rate-limiting windows are durations, not dates."

---

6. Generic algorithm versus runtime owner

The design separates algorithm evaluation from the object that owns live state:

- `RateLimitAlgorithm<S, C>` describes an algorithm over state type `S` and configuration type `C`.
- `FixedWindowAlgorithm` implements the fixed-window rule.
- An evaluation result describes the decision and the next state.
- `RateLimiter` owns the active runtime state and serializes evaluation/commit.

Why can this be useful?

The algorithm becomes easier to test in isolation. Runtime coordination such as mutexes and current-state ownership does not have to be mixed into the algorithm's rule calculations. The abstraction also leaves room to explore other algorithms later.

Trade-off

Generics and interfaces introduce indirection. For one tiny fixed-window app, a concrete class could be simpler. The abstraction is justified here mainly because the project is a learning playground for algorithm comparison and testing—not because every small app needs a generic algorithm framework.

7. Coordinator, configuration replacement, and reset semantics

The `RateLimiterCoordinator` owns the active limiter and current configuration. It serializes operations that select/replace/reset the active limiter.

Configuration replacement

The project replaces the active limiter when a new configuration is applied. This gives the new configuration a fresh runtime state rather than trying to reinterpret the old window/count state under new rules.

A useful concurrency detail: if a request has already selected the old limiter before the coordinator swaps the active reference, that in-flight request may finish on the old limiter. Requests that select the active limiter after the swap should use the new limiter. This is a clear, implementable boundary—though it is not the same as cancelling every old in-flight request.

Reset semantics

Reset creates/replaces the active limiter with the current configuration and fresh runtime state. The UI returns to an idle request state while preserving the applied configuration and draft input fields, unless product requirements say otherwise.

Interview question: Why replace the limiter instead of mutating it?

"Replacing the limiter provides a clean state boundary. It avoids needing to transform existing runtime state when configuration changes. The trade-off is that in-flight work already holding the old limiter can finish there; the coordinator's selection/swap semantics need to be explicit and tested."

8. ViewModel state, MVI-style intents, and StateFlow

Key concepts

- `RateLimiterIntent` describes actions such as `SimulateRequest`, `Reset`, `ApplyConfiguration`, and text-input edits.
- `RateLimiterUiState` is an immutable snapshot of what the UI should render: active configuration, draft input values, request status, and validation error.
- `RequestUiState` models request status, such as `Idle`, `Processing`, and `Completed(decision)`.
- A private `MutableStateFlow` is updated by the ViewModel; the UI receives a read-only `StateFlow` via `asStateFlow()`.
- Compose collects the state lifecycle-aware and renders it. Buttons/text fields dispatch intents rather than reaching into the coordinator directly.

Why state is immutable

Publishing a new state value makes UI transitions explicit and predictable. `StateFlow` exposes the latest state and emits updated values to collectors. Compose can render from that state instead of duplicating business state internally.

Why validation belongs in the ViewModel here

The ViewModel validates user-entered strings before converting them into a domain configuration. This keeps text-field drafts separate from the validated `RateLimitConfiguration` and ensures the UI cannot bypass validation by directly constructing a configuration intent. For a larger app, validation rules that are domain-critical might be placed in a domain use case/value object so non-UI callers also benefit.

Why use `CancellationException` correctly?

When catching broad exceptions around coroutine work, rethrow `CancellationException`. Cancellation is part of coroutine control flow, not an ordinary recoverable error. Swallowing it can break structured cancellation and make jobs continue when their parent has been cancelled.

---

9. Version counters and stale results

The ViewModel uses version counters for distinct concerns:

- **`configurationVersion`**: invalidates request results associated with an older configuration/reset boundary.
- **`configurationOperationVersion`**: helps determine which pending Apply/Reset operation is the latest intent.
- **`draftVersion`**: detects if the user edited text fields while an apply operation was running, so the completed operation doesn't overwrite newer draft input.
- **`Mutex`**: serializes protected asynchronous operations. A version counter does not replace a mutex, and a mutex does not automatically invalidate stale UI results.

Why use both a mutex and versions?

A mutex answers, "Which operation may execute its protected section now?" A version check answers, "Is this result still relevant enough to publish?" They protect different invariants.

Important limitation to explain honestly

Version counters are a deliberate solution for this playground, not a universal state-management pattern. Their correctness depends on updating/checking them consistently and clearly defining when an operation becomes effective. A more complex app might instead model commands with a single actor/channel, serialize all intents in one reducer, or use explicit operation jobs. Avoid adding counters without describing exactly what each counter invalidates.

Also distinguish these scenarios when discussing tests:

- An older operation that is still queued can often be skipped after a newer intent arrives.
- An older operation that has already changed the coordinator generally cannot be undone; it may finish first and the newer operation then applies afterward.
- An in-flight request may have completed domain work even if the ViewModel suppresses its stale UI result.

---

10. Dependency Injection and Hilt

Dependency injection (DI) means an object receives its dependencies from outside rather than constructing them internally. Constructor injection is the main pattern used in this project.

Why DI helps

- The ViewModel depends on `RateLimiterCoordinator`, not on how it is constructed.
- Tests can supply controlled dependencies without launching the Android app or Hilt container.
- Production implementations can be replaced by test fakes or alternative implementations.
- Construction policy is separated from business behavior.

Hilt annotations to remember

- `@HiltAndroidApp`: initializes Hilt's application-level component; place it on the custom `Application` class registered in the manifest.
- `@AndroidEntryPoint`: enables Hilt injection support for an Android entry point such as the Activity.
- `@HiltViewModel` and `@Inject constructor`: allow Hilt to provide the ViewModel's constructor dependencies.
- `@Module`: groups dependency definitions.
- `@Binds`: maps an abstraction to an implementation, such as `TimeProvider` to `AndroidTimeProvider`.
- `@Provides`: contains construction logic for a dependency that needs custom creation, such as a default `RateLimitConfiguration`.
- `@InstallIn`: chooses which Hilt component owns the bindings.

`@Binds` versus `@Provides`

Use `@Binds` when the binding is simply "this interface is implemented by that injectable class." Use `@Provides` when code is needed to construct the value or choose specific arguments. A Kotlin `object` is a common module form for `@Provides`; an abstract class is a common module form for `@Binds`. These are normal patterns, not an absolute statement that modules must always have those exact Kotlin forms.

Scope is not the same as singleton module syntax

A Kotlin `object` module does not make every provided object a singleton. Object lifetime is determined by the Hilt binding and its component/scope. Scopes should be chosen based on the intended lifecycle—for example, per-ViewModel or application-wide—not added automatically.

Why is `RateLimiterApplication` needed?

Android creates the registered `Application` instance. The class annotated with `@HiltAndroidApp` lets generated Hilt code initialize the application component. You don't manually call its constructor.

Important design question: where should initial configuration live?

The project currently provides a default initial configuration for this playground. In a production app, configuration may be user-controlled, environment-specific, loaded from persistence, or supplied by a use case. Avoid treating a hard-coded `@Provides` value as the right approach for all apps. If the user edits configuration at runtime, that edit is runtime state managed by the coordinator/ViewModel; Hilt's initial binding is not a live configuration store.

---

11. Unit testing strategy

The project uses different test levels because no single test type proves everything.

Algorithm tests

Test rules and boundaries directly: allowed/rejected decisions, window expiry, retry-after behavior, and configuration validation. These tests should be fast and deterministic.

Concurrency tests

Launch overlapping requests and assert invariants, such as exactly one allowed request under a limit of one for a fixed window. Use synchronization tools in the test to coordinate the overlap instead of relying on arbitrary delays.

Coordinator tests

Verify active limiter replacement, reset semantics, and the expected behavior of an in-flight request across a configuration change. Use explicit barriers/gates where the test must hold an operation at a known point.

ViewModel tests

Use a fake/test clock and a test dispatcher for `Dispatchers.Main` because `viewModelScope` uses Main. `MainDispatcherRule` plus `StandardTestDispatcher` allows the test to run scheduled coroutine work deterministically; `advanceUntilIdle()` processes pending tasks.

Tests practiced in this project include:

- Invalid limit input displays an error and preserves the active configuration.
- A simulated request transitions through `Processing` and to a completed decision.
- Reset returns request UI state to idle and preserves the active configuration.
- Draft text edits made while Apply is pending are preserved.
- A newer pending Apply can supersede an older queued Apply.

Why don’t normal ViewModel unit tests need Hilt?

The ViewModel has constructor-injected dependencies, so the test can construct it directly and pass a real lightweight coordinator or a fake dependency. Hilt integration tests are useful when testing Hilt wiring itself, but using the full DI container for every local unit test adds setup cost without improving the test’s focus.

Test quality caveat

A test that submits two operations to a test dispatcher and advances until idle may prove queued-operation supersession, but it does not automatically prove every race involving an operation already running or suspended inside the coordinator. Describe precisely what a test coordinates and what invariant it checks. For important concurrency behavior, consider a gate-controlled fake or explicit test hook that pauses an operation at a known point.

---

12. Common interview questions and strong answer outlines

Q1. Why use a `Mutex` in the rate limiter?

Because reading state, evaluating the request, and committing new state form one critical section. Without serialization, concurrent requests could evaluate against the same old state and both be accepted.

Q2. Why inject a clock?

To isolate platform time access and make time-dependent tests deterministic. A fake clock can move directly across a window boundary without sleeping.

Q3. Why use monotonic time?

The algorithm measures elapsed durations, not calendar timestamps. Wall-clock adjustments should not change the elapsed-time calculation.

Q4. Why use `StateFlow` for UI state?

It represents a hot stream with a current value and lets the UI observe immutable state snapshots. Compose renders from the ViewModel’s state instead of maintaining a second copy of application behavior.

Q5. Why use sealed types for intents and request state?

They make possible actions/statuses explicit and allow exhaustive `when` handling. This makes missing cases visible at compile time and avoids loosely related Boolean flags.

Q6. Why have a coordinator in addition to the limiter?

The limiter enforces one limiter’s algorithm/state. The coordinator manages which limiter is active and coordinates replacement/reset. Separating them gives configuration lifecycle a clear owner.

Q7. Why replace a limiter when configuration changes?

It creates a clear state boundary and avoids interpreting old runtime state under new parameters. In-flight operations that already selected the old limiter may finish there, so the semantics must be explicit.

Q8. Why Hilt instead of manually creating dependencies?

Hilt automates construction and wiring for the Android application. The underlying DI principle is still constructor injection; Hilt is a tool rather than the architectural principle itself.

Q9. Do unit tests need Hilt?

Usually not. Construct the ViewModel directly and supply dependencies. Use Hilt's testing support only when the dependency graph/injection wiring itself is part of what you need to verify.

Q10. What is one limitation of this project?

The in-memory fixed-window limiter is not distributed and can allow boundary bursts. The app focuses on exploring concurrency, Android architecture, and testability rather than implementing a production-wide rate-limiting service.

---

13. Things to avoid saying in an interview

- Don't say a `Mutex` makes the entire app thread-safe. It only protects code using that specific mutex.
- Don't say fixed-window limiting strictly bounds requests in every rolling interval; boundary bursts are a known limitation.
- Don't say Hilt is required for dependency injection or for ordinary ViewModel unit tests.
- Don't say `@Provides` makes its result singleton just because the module is a Kotlin `object`.
- Don't claim a test proves all concurrency races if it only exercises queued coroutines.
- Don't call every layered project "Clean Architecture" without explaining the actual dependency boundaries.
- Don't add abstractions just to use a pattern. Explain what complexity or testing need each abstraction addresses.

---

14. Final project review checklist

Before presenting this project in an interview, verify the checked-in version matches the claims in this guide:

- ☐ The app builds and launches from a clean checkout.
- ☐ Unit tests pass.
- ☐ The fixed-window algorithm tests include boundaries and rejection behavior.
- ☐ The concurrency test produces overlap and checks the allowed/rejected invariant.
- ☐ Coordinator tests document what happens to in-flight requests during replacement/reset.
- ☐ ViewModel tests use a deterministic coroutine test dispatcher.
- ☐

Hilt bindings match the actual constructor signatures and scopes.

- [ ] `RateLimiterApplication` is declared in the manifest.
- [ ] `MainActivity` uses Hilt-enabled ViewModel retrieval rather than the old manual factory.
- [ ] No secrets, local machine files, or generated artifacts are committed.
- [ ] The README explains how to build and test the project.

15. Suggested README section

You can add this short summary to the project's `README.md`:

> **What this project demonstrates:** fixed-window rate limiting, coroutine-safe state transitions, monotonic time abstraction, configuration replacement semantics, unidirectional UI state updates with `StateFlow`, constructor injection with Hilt, and deterministic unit testing. The implementation is intentionally small so the concurrency invariants and architectural trade-offs are easy to inspect.

Main takeaway

The biggest lesson is not "use Hilt," "use `Mutex`," or "use StateFlow." It is to **define invariants, give mutable state a clear owner, make time and dependencies replaceable, and test the behavior at the right level**. In a senior interview, explain the problem each mechanism solves, the alternatives you considered, and the limitations that remain.